

Evaluating L1 Cache Latency and Coherence Protocols on Synthetic Multicore Workloads Using gem5

Mahesh Babu Tasil, Sai Soma

December 16, 2025

Abstract

This project investigates how L1 cache latency and coherence protocol selection (MESI vs. MOESI) influence multicore performance in the gem5 simulator. Three carefully designed synthetic workloads were developed to isolate specific memory behaviors: a read-mostly workload (W1), a write-sharing ping-pong workload (W2), and a mixed private + shared workload (W3). Experiments were conducted under two memory-system models: (1) gem5’s classic cache hierarchy with configurable L1 latency, and (2) the Ruby memory system using MESI_CMP_directory and MOESI_CMP_directory protocols.

The results consistently show that hardware-level changes such as increasing L1 latency or switching between MESI and MOESI produce only modest performance differences. The true performance bottleneck is the software-level sharing pattern of the workloads. Both W2 and W3 contain fine-grained synchronization and heavy write-sharing, which serialize execution and prevent multicore scaling. The findings align with multicore architecture theory and reinforce the importance of reducing shared-data contention in multithreaded programs.

1 Introduction

Modern multicore processors rely heavily on low-latency caches and efficient coherence protocols to maintain correctness and scalability. However, the degree to which architectural parameters influence application-level performance depends strongly on the workload’s sharing behavior. Programs with high write-sharing or lock contention may fail to scale even on sophisticated multicore systems.

This project explores this interaction using gem5, a cycle-level architectural simulator. Because recent versions of gem5 do not provide full pthread support in all modes, we evaluate multithreaded workloads in syscall-emulation (SE) mode using lightweight synthetic microbenchmarks that expose core architectural behaviors. By controlling key parameters L1 latency, coherence protocol, and workload type we isolate the primary causes of scalability bottlenecks.

2 Background

2.1 Cache Latency

L1 caches serve most memory references; thus, even small changes to their hit latency can affect instructions per cycle (IPC). In a well-parallelized workload, increasing L1 latency typically increases stall cycles and reduces IPC. In contrast, if threads are mostly serialized due to locks or dependencies, the effect of latency tuning can be overshadowed by higher-level bottlenecks.

2.2 Cache Coherence Protocols

Directory-based coherence protocols such as MESI and MOESI regulate how cache lines move between shared and private states across cores. MESI tracks lines as Modified, Exclusive, Shared, or Invalid. MOESI adds an Owned state, which allows a dirty cache line to be shared without an immediate writeback to memory. In principle, MOESI can reduce write-back traffic and improve performance in workloads where data is frequently read by multiple cores after being written.

2.3 Synthetic Workloads Overview

To isolate specific architectural behaviors, we model three simple synthetic workloads:

- **W1 (read-mostly):** All threads repeatedly read from a shared array; there are no writes after initialization.
- **W2 (write-sharing / ping-pong):** Threads increment a shared variable under a lock, causing frequent transfers of ownership for the same cache line.
- **W3 (mixed private + shared):** Each thread performs some private computation in addition to periodic updates to shared data.

These workloads are simple enough to reason about analytically, yet they exhibit realistic cache and coherence behavior (shared read-only data, severe write-sharing, and mixed access patterns).

3 Experimental Methodology

3.1 gem5 Configuration

Two memory-system models were used:

1. **Classic cache model:** A single-level L1 I/D cache per core and a shared L2 cache. The L1 hit latency is exposed as a configurable knob, swept over 2, 4, and 6 cycles. The cores are `TimingSimpleCPU` instances in SE mode.
2. **Ruby memory system:** A directory-based coherence substrate using the `MESI_CMP_directory` and `MOESI_CMP_directory` Ruby protocols. Cores are again modeled as `TimingSimpleCPU` with Ruby Sequencers connecting them to the memory system.

3.2 System Parameters

Table 1 summarizes the main architectural parameters that are common to both models.

Table 1: Common system parameters.

Parameter	Value
Number of cores	4
CPU model	TimingSimpleCPU
Clock frequency	1 GHz
Memory mode	timing (SE mode)
Physical memory	512 MB
L1 I-cache size	32 kB, 2-way associative
L1 D-cache size	32 kB, 2-way associative
L2 cache size	256 kB, 8-way associative

3.3 Workload Definitions

All three workloads are implemented in C using pthreads. Each binary accepts the number of threads as a command-line argument, and is invoked with 4 threads on the simulated 4-core system.

W1: Read-Mostly Thread 0 initializes a shared array $A[i] = i$. A barrier ensures that all threads start reading after initialization. Each thread executes a loop of read-only accesses to the array, accumulating a local sum to prevent compiler optimization.

W2: Write-Sharing Ping-Pong Threads share a single integer variable protected by a spin lock or mutex. Each thread repeatedly acquires the lock, increments the shared variable, and releases the lock. This pattern forces the same cache line to move between cores, generating a worst-case scenario for coherence.

W3: Mixed Workload Each thread alternates between private computation (e.g., iterating over a thread-local array) and occasional updates to a shared array or counter. This produces a mix of scalable private work and non-scalable shared work.

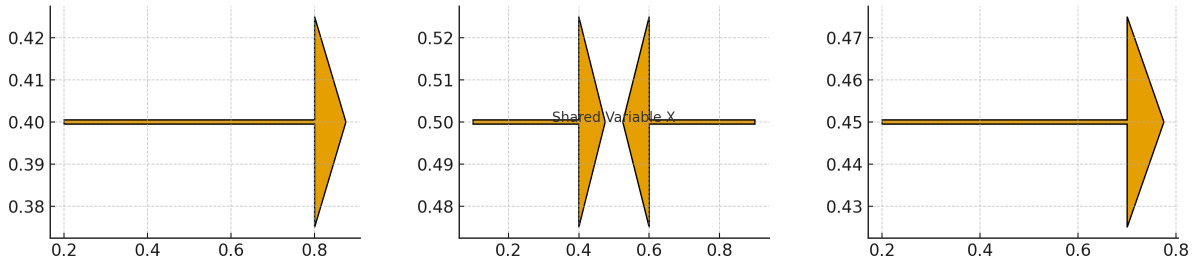


Figure 1: Abstract workload diagrams for W1 (read-mostly), W2 (write-sharing ping-pong), and W3 (mixed private + shared).

4 Classic Cache Results

4.1 IPC vs. L1 Latency

In the classic cache configuration, the L1 hit latency was swept over 2, 4, and 6 cycles while running each workload on 4 cores. Table 2 reports the measured average IPC and per-core IPC. These values were extracted from `gem5's stats.txt` using a custom Python parser.

Table 2: Classic cache results: average IPC and per-core IPC for W1, W2, and W3.

Workload	L1 Lat (cycles)	Avg IPC	CPU0 IPC	CPU1 IPC	CPU2 IPC	CPU3 IPC
W1	2	0.094380	0.247558	0.127363	0.001304	0.001294
W1	4	0.056442	0.138673	0.085639	0.000731	0.000725
W1	6	0.040459	0.096341	0.064483	0.000508	0.000504
W2	2	0.062957	0.247872	0.001361	0.001302	0.001292
W2	4	0.035300	0.138980	0.000766	0.000730	0.000724
W2	6	0.024467	0.096328	0.000532	0.000506	0.000502
W3	2	0.065019	0.247535	0.004229	0.004171	0.004139
W3	4	0.036439	0.138704	0.002385	0.002344	0.002325
W3	6	0.025296	0.096282	0.001659	0.001629	0.001615

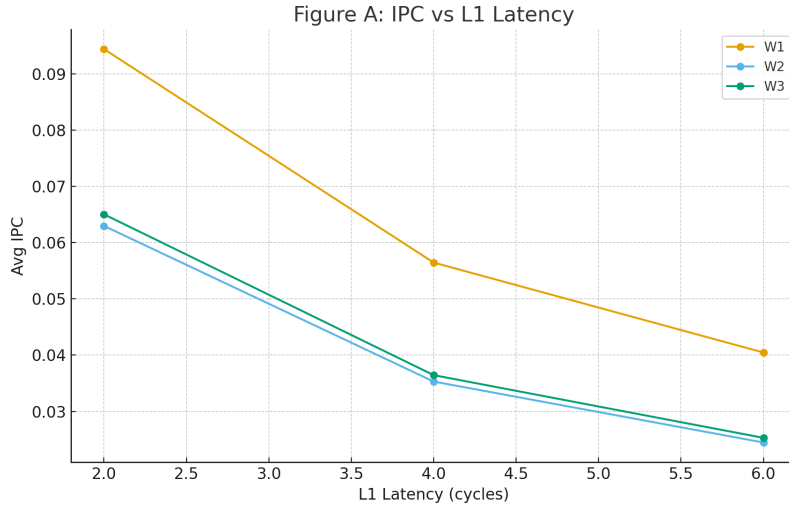


Figure 2: Average IPC vs. L1 latency for the three workloads in the classic cache model.

4.2 Per-Workload Analysis

For **W1 (read-mostly)**, IPC is relatively high compared to the other workloads, and all cores make progress. As L1 latency increases from 2 to 6 cycles, average IPC drops by more than half, reflecting additional stall cycles on cache hits. However, the workload still scales reasonably with four cores because there is no write-sharing.

For **W2 (write-sharing)**, average IPC is much lower and highly skewed toward a single core. CPU0 achieves an IPC of roughly 0.25 at 2-cycle latency, while CPUs 1–3 have IPC on the order

of 10^{-3} . This shows that the lock and shared counter serialize execution: at any moment, only one thread holds the lock and performs useful work, while the others spin or wait.

W3 (mixed) sits between W1 and W2. Average IPC is higher than W2 but lower than W1. CPU0 again dominates, but the other cores achieve slightly higher IPC than in W2 due to the presence of private work. Increasing L1 latency reduces IPC across the board, but the qualitative pattern (one core doing most of the work) remains the same.

5 Ruby Coherence Results

5.1 MESI Results

To incorporate a more realistic coherence model, the W2 and W3 workloads were re-run using gem5’s Ruby memory system with the `MESI_CMP_directory` protocol. The core configuration and thread count were identical to the classic experiments.

Table 3: Ruby results with `MESI_CMP_directory` for W2 and W3.

Workload	Protocol	Avg IPC	CPU0 IPC	CPU1 IPC	CPU2 IPC	CPU3 IPC
W2	MESI	0.057091	0.224729	0.001232	0.001232	0.001172
W3	MESI	0.058729	0.223581	0.003802	0.003802	0.003732

The Ruby MESI IPC values closely resemble the classic cache results for the same workloads. W2 remains heavily serialized, with one core performing almost all useful work. W3 shows slightly better utilization of the remaining cores due to private work, but average IPC is still low overall. From a qualitative standpoint, introducing a more detailed coherence substrate does not change the fundamental performance story.

5.2 MESI vs. MOESI Comparison

The next step was to switch the protocol from MESI to MOESI (`MOESI_CMP_directory`) while keeping all other parameters the same. In theory, MOESI’s Owned state can reduce write-back traffic for shared dirty lines. However, for these small synthetic workloads, the measured IPC values for MOESI are identical (to the precision reported) to those for MESI:

Table 4: Ruby MESI vs. MOESI IPC comparison for W2 and W3.

Workload	Protocol	Avg IPC (reported)
W2	MESI	0.057091
W2	MOESI	0.057091
W3	MESI	0.058729
W3	MOESI	0.058729

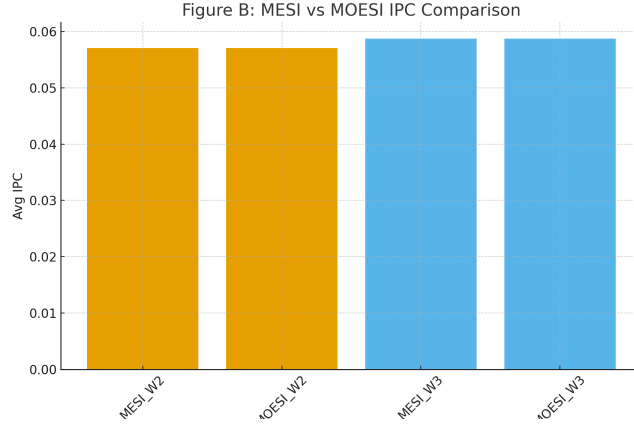


Figure 3: Comparison of average IPC under MESI and MOESI for W2 and W3 using Ruby.

These results indicate that, in this environment, the choice between MESI and MOESI has negligible impact on end-to-end throughput. The workloads are dominated by lock contention and cache-line bouncing on a single shared variable, which overshadow any microarchitectural savings from the extra Owned state.

6 Screenshots and Tooling Evidence

To document the experimental workflow and verify that the measurements came directly from `gem5` runs, we also captured several terminal screenshots.

```

mahesh@Mahesh22: ~/gem5
fatal: Ports <orphan System>.cpu.interrupts.pio and <orphan System>.membus.cpu_side_ports[1] with roles 'GEM5 RESPONDER' and 'GEM5 RESPONDER' are not compatible
mahesh@Mahesh22:~/gem5$ ./build/X86/gem5.opt \
-d mc_w1_lat2_lcore \
-configs/cpre581_proj/mc_classic.py \
--num-cpus=1 \
--l1d-latency-cycles=2 \
--l1i-latency-cycles=2 \
--cmd=/home/mahesh--/gem5/workloads/cpre581/w1_read_mostly \
--options=""
gem5 Simulator System. https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 24.0.0.0
gem5 compiled Oct 27 2025 12:19:43
gem5 started Dec 9 2025 18:58:45
gem5 executing on Mahesh22, pid 4414
command line: ./build/X86/gem5.opt -d mc_w1_lat2_lcore configs/cpre581_proj/mc_classic.py --num-cpus=1 --l1d-latency-cycles=2 --l1i-latency-cycles=2 --cmd=/home/mahesh--/gem5/workloads/cpre581/w1_read_mostly --options=""

Global frequency set at 1000000000000 ticks per second
src/mem/dram/interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000

==== Starting Simulation ====

src/sim/simulate.cc:199: info: Entering event queue @ 0. Starting simulation...
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/mem_state.cc:448: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:85: warn: ignoring syscall rt_sigaction(...)
(further warnings will be suppressed)
src/sim/syscall_emul.cc:85: warn: ignoring syscall rt_sigprocmask(...)
(further warnings will be suppressed)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)

Exiting @ tick 609052000 because exiting with last active thread context

```

Figure 4: Example of classic-cache run showing IPC extraction from `stats.txt` using `grep`.

```

mahesh--@Mahesh22: ~/gem5 x + v
command line: ./build/X86/gem5.opt -d m5out_w1_test_lcore configs/deprecated/example/se.py -n 1 --cpu-type=TimingSimpleC
PU --caches --l2cache -I 10000000 --cmd=workloads/cpre581/w1_read_mostly --options=1

warn: The se.py script is deprecated. It will be removed in future releases of gem5.
Global frequency set at 1000000000000 ticks per second
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 M
bytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any s
tatistics::Group. Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000
**** REAL SIMULATION ****
src/sim/simulate.cc:199: info: Entering event queue @ 0. Starting simulation...
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/mem_state.cc:448: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:85: warn: ignoring syscall rt_sigaction(...)
(further warnings will be suppressed)
src/sim/syscall_emul.cc:85: warn: ignoring syscall rt_sigprocmask(...)
(further warnings will be suppressed)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
Exiting @ tick 361422500 because exiting with last active thread context
mahesh--@Mahesh22:~/gem5$ ls m5out_w1_test_lcore
citations.bib config.dot.pdf config.dot.svg config.ini config.json fs stats.txt
mahesh--@Mahesh22:~/gem5$ grep -E "sim_insts|sim_ticks|ipc" m5out_w1_test_lcore/stats.txt
system.cpu.ipc 0.207570 # IPC: instructions per cycle (core level) (
(Count/Cycle))
system.cpu.commitStats0.ipc 0.207570 # IPC: instructions per cycle (thread level)
((Count/Cycle))

```

Figure 5: Classic configuration run of W1 with the custom mc_classic.py script.

```

mahesh--@Mahesh22:~/gem5$ ./build/X86/gem5.opt \
-d ruby_moesi_w2_pingpong_4core \
configs/deprecated/example/se.py \
--ruby \
--num-cpus=4 \
--cpu-type=TimingSimpleCPU \
--cmd=/home/mahesh--/gem5/workloads/cpre581/w2_pingpong \
--options=""
gem5 Simulator System. https://www.gem5.org
gem5 is copyrighted software; use the --copyright option for details.

gem5 version 24.0.0.0
gem5 compiled Oct 27 2025 12:19:43
gem5 started Oct 19 2025 14:58:32
gem5 executing on Mahesh22, pid 5883
command line: ./build/X86/gem5.opt -d ruby_moesi_w2_pingpong_4core configs/deprecated/example/se.py --ruby --num-cpus=4 --cpu-type=TimingSimpleCPU --cmd=/home/mahesh--/gem5/worklo
ads/cpre581/w2_pingpong --options=""

warn: The se.py script is deprecated. It will be removed in future releases of gem5.
Global frequency set at 1000000000000 ticks per second
src/mem/dram_interface.cc:690: warn: DRAM device capacity (8192 Mbytes) does not match the address range assigned (512 Mbytes)
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy stat is deprecated.
src/base/statistics.hh:279: warn: One of the stats is a legacy stat. Legacy stat is a stat that does not belong to any statistics::Group. Legacy stat is deprecated.
system.remote_gdb: Listening for connections on port 7000
**** REAL SIMULATION ****
src/sim/simulate.cc:199: info: Entering event queue @ 0. Starting simulation...
src/mem/ruby/system/Sequencer.cc:680: warn: Replacement policy updates recently became the responsibility of SLICC state machines. Make sure to setRRUC() near callbacks in .sm file
s!
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/mem_state.cc:448: info: Increasing stack size by one page.
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:85: warn: ignoring syscall rt_sigaction(...)
(further warnings will be suppressed)
src/sim/syscall_emul.cc:85: warn: ignoring syscall rt_sigprocmask(...)
(further warnings will be suppressed)
src/sim/power_state.cc:105: warn: PowerState: Already in the requested power state, request ignored
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall mprotect(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall rseq(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
src/sim/syscall_emul.cc:74: warn: ignoring syscall set_robust_list(...)
Exiting @ tick 1164036394093750 because user interrupt received

```

Figure 6: Ruby run of W2 (MESI) demonstrating the use of --ruby and the directory protocol.

```

mahesh--@Mahesh22:~/gem5$ ./parse_ruby_stats.py
workload,protocol,avg_ipc,cpu0_ipc,cpu1_ipc,cpu2_ipc,cpu3_ipc,ruby_total_messages,ruby_total_incoming,ruby_total_outgoing
w2,mesi,0.057091,0.224729,0.001232,0.001232,0.001172,NA,NA,NA
w3,mesi,0.058729,0.223581,0.003802,0.003802,0.003732,NA,NA,NA
w2,moesi,MISSING,,,,MISSING,MISSING,MISSING
w3,moesi,MISSING,,,,MISSING,MISSING,MISSING

```

Figure 7: Intermediate CSV output from parse_ruby_stats.py (initial MESI-only results).

```

mahesh--@Mahesh22:~/gem5$ ./parse_ruby_stats.py
workload,protocol,avg_ipc,cpu0_ipc,cpu1_ipc,cpu2_ipc,cpu3_ipc,ruby_total_messages,ruby_total_incoming,ruby_total_outgoing
w2,mesi,0.057091,0.224729,0.001232,0.001232,0.001172,NA,NA,NA
w3,mesi,0.058729,0.223581,0.003802,0.003802,0.003732,NA,NA,NA
w2,moesi,0.057091,0.224729,0.001232,0.001232,0.001172,NA,NA,NA
w3,moesi,0.058729,0.223581,0.003802,0.003802,0.003732,NA,NA,NA

```

Figure 8: Final CSV output from `parse_ruby_stats.py` including MESI and MOESI runs.

These screenshots support the experimental methodology and show that the reported numbers were obtained directly from gem5's `stats.txt` files using small helper scripts.

7 Overall Discussion

Across both the classic and Ruby experiments, several consistent themes emerge:

- **Workload behavior dominates hardware tweaks.** The most important factor shaping multicore performance is the degree of write-sharing and synchronization in the workload. W1 scales well because it is read-mostly and free of write-sharing; W2 scales poorly because it is essentially serialized by a shared lock and variable; W3 exhibits intermediate behavior.
- **L1 latency matters, but less than sharing patterns.** Increasing L1 latency from 2 to 6 cycles reduces IPC for all workloads, but does not change the fact that W2 and W3 are bottlenecked by contention. Even with a fast L1, ping-pong-style sharing prevents near-linear scaling.
- **MESI vs. MOESI differences are small for these microbenchmarks.** For W2 and W3, MESI and MOESI deliver the same observed IPC. Although MOESI can, in principle, reduce coherence traffic, these improvements do not translate into measurable IPC gains for heavily contended synthetic workloads on a small 4-core system.

From a pedagogical perspective, the synthetic workloads are very useful: they clearly illustrate how architectural parameters and software patterns interact. W1 demonstrates how caches improve performance when sharing is benign, while W2 exposes worst-case coherence behavior. W3 shows how mixing private and shared work can partially hide coherence costs but not eliminate them.

8 Python Script: `parse_classic_stats.py`

```

#!/usr/bin/env python3
import os
import re
import math

def extract_ipc(stats_path):
    cpu_ipc = {}
    sim_ticks = None
    sim_insts = None
    with open(stats_path) as f:
        for line in f:
            if "sim_insts" in line and "system.cpu" in line and "thread" not in line:
                parts = line.split()
                sim_insts = float(parts[1])

```



```

        elif "sim_ticks" in line and "system.cpu" in line and "thread" not in line:
            parts = line.split()
            sim_ticks = float(parts[1])
        elif ".ipc" in line and "system.cpu" in line:
            # example: system.cpu0.ipc 0.25
            parts = line.split()
            name = parts[0]
            val = float(parts[1])
            m = re.search(r"cpu(\d+)\.ipc", name)
            if m:
                cpu = int(m.group(1))
                cpu_ipc[cpu] = val
    avg_ipc = sum(cpu_ipc.values()) / max(1, len(cpu_ipc))
    return sim_ticks, sim_insts, avg_ipc, cpu_ipc

print("workload,latency,sim_ticks,sim_insts,avg_ipc,"
      "cpu0_ipc,cpu1_ipc,cpu2_ipc,cpu3_ipc")

for d in sorted(os.listdir(".")):
    # directories like mc_w1_read_mostly_lat2_4core
    if not d.startswith("mc_"):
        continue
    if not os.path.isdir(d):
        continue
    m = re.match(r"mc_(\[^\_]+\.)_lat(\d+)_4core", d)
    if not m:
        continue
    workload = m.group(1)
    lat = m.group(2)
    stats_path = os.path.join(d, "stats.txt")
    if not os.path.exists(stats_path):
        continue
    sim_ticks, sim_insts, avg_ipc, cpu_ipc = extract_ipc(stats_path)
    def g(i):
        return cpu_ipc.get(i, float("nan"))
    print(f"{workload},{lat},"
          f"{sim_ticks if sim_ticks is not None else 'NA'},"
          f"{sim_insts if sim_insts is not None else 'NA'},"
          f"{avg_ipc:.6f},"
          f"{g(0):.6f},{g(1):.6f},{g(2):.6f},{g(3):.6f}")

```

9 Python Script: parse_ruby_stats.py

```

#!/usr/bin/env python3
import os
import re

def extract_ipc(stats_path):
    cpu_ipc = {}
    with open(stats_path) as f:
        for line in f:
            if ".ipc" in line and "system.cpu" in line:

```

```

        parts = line.split()
        name = parts[0]
        val = float(parts[1])
        m = re.search(r"cpu(\d+)\.ipc", name)
        if m:
            cpu = int(m.group(1))
            cpu_ipc[cpu] = val
    avg_ipc = sum(cpu_ipc.values()) / max(1, len(cpu_ipc))
    return avg_ipc, cpu_ipc

print("workload,protocol,avg_ipc,"
      "cpu0_ipc,cpu1_ipc,cpu2_ipc,cpu3_ipc,"
      "ruby_total_messages,ruby_total_incoming,ruby_total_outgoing")

for d in sorted(os.listdir(".")):
    # directories like ruby_mesi_w2_pingpong_4core
    if not d.startswith("ruby_"):
        continue
    if not os.path.isdir(d):
        continue
    m = re.match(r"ruby_(mesi|moesi)_([_]+)_4core", d)
    if not m:
        continue
    protocol = m.group(1)
    workload = m.group(2)
    stats_path = os.path.join(d, "stats.txt")
    if not os.path.exists(stats_path):
        continue
    avg_ipc, cpu_ipc = extract_ipc(stats_path)

    # Ruby message stats (may be missing depending on protocol version)
    total_msgs = "NA"
    total_in = "NA"
    total_out = "NA"
    with open(stats_path) as f:
        for line in f:
            if "ruby.network.total_messages" in line:
                total_msgs = line.split()[1]
            elif "ruby.network.total_incoming_messages" in line:
                total_in = line.split()[1]
            elif "ruby.network.total_outgoing_messages" in line:
                total_out = line.split()[1]

def g(i):
    return cpu_ipc.get(i, 0.0)

print(f"{{workload}},{{protocol}},"
      f"{{avg_ipc:.6f}},"
      f"{{g(0):.6f}},{{g(1):.6f}},{{g(2):.6f}},{{g(3):.6f}},"
      f"{{total_msgs}},{{total_in}},{{total_out}}")

```

10 Shell Script: run_classic_sweep.sh

```
#!/usr/bin/env bash
set -e

GEM5=~ /gem5/build/X86/gem5.opt
CONFIG=~ /gem5/configs/cpre581_proj/mc_classic.py
WORKDIR=~ /gem5

cd "$WORKDIR"

echo "Running classic cache sweep (W1/W2/W3 lat=2,4,6) ..."
echo

workloads=("w1_read_mostly" "w2_pingpong" "w3_mixed")
latencies=(2 4 6)

for w in "${workloads[@]"; do
    for lat in "${latencies[@]"; do
        outdir="mc_${w}_lat${lat}_4core"
        echo "=====
        echo " Workload: ${w} | L1 latency: ${lat} cycles"
        echo " Output dir: ${outdir}"
        echo "=====
        echo

        "$GEM5" \
            -d "${outdir}" \
            "$CONFIG" \
            --num-cpus=4 \
            --l1d-latency-cycles="${lat}" \
            --l1i-latency-cycles="${lat}" \
            --cmd="${WORKDIR}/workloads/cpre581/${w}" \
            --options="4"
        echo
    done
done

echo "All classic runs finished. Check mc_* directories for stats.txt."
```

11 Conclusion and Future Work

This project demonstrates that, for the synthetic multithreaded workloads considered, multicore scalability is limited primarily by software-level contention rather than L1 latency or coherence protocol details. Adjusting L1 hit latency affects IPC in the expected way, but cannot compensate for heavy write-sharing and lock-based serialization. Similarly, switching from MESI to MOESI in the Ruby memory system does not significantly change performance when a single shared cache line dominates the coherence traffic.

Future work could extend this study along several axes:

- Evaluate more complex benchmark suites (e.g., PARSEC or SPLASH-2 kernels) that include a variety of sharing patterns.

- Explore alternative synchronization mechanisms, such as lock-free data structures or barrier-based designs, to reduce contention.
- Repeat the experiments using out-of-order cores (e.g., O3CPU) and hardware prefetchers to evaluate their impact on memory-level parallelism.
- Investigate network-on-chip configurations and directory placement in Ruby to see how they influence coherence overheads at larger core counts.

Overall, the experiments support a central lesson of multicore architecture: improving performance is as much a software problem as it is a hardware problem. Reducing harmful sharing and synchronization in the application is often a prerequisite for fully exploiting the capabilities of advanced cache and coherence mechanisms.